Software Architecture and Design Specification

Project: Hospital Management System (HMS)

Version: 1.0 (Final – Project Baseline)

Authors: Hrushikesh Karnam (PES1UG24CS668), Muruli H (PES1UG24CS676),Mandagandla Jhansi (PES1UG24CS672), Neeta R (PES1UG24CS680)

Date: 28-09-2026

Status: Final – Project Baseline v1.0

Course: Software Engineering Mini-Project, Department of Computer Science and Engineering

Revision History

Version	Date	Author	Change Summary
1.0	28-09-2026	HMS Project Team	Initial baseline SAD derived from SRS v1.0 and STP v1.0

Approvals

Role	Name	Signature / Date
Course Coordinator / Evaluator		
Project Team Representative		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Hospital Management System (HMS) v1.0. It explains how the requirements of the HMS Software Requirements Specification (SRS) v1.0 are realised in a set of components, classes, interfaces and data files, and it is the reference for implementation and for the Software Test Plan (STP).

1.2 Scope

HMS is a standalone, single-user C++17 console application used by hospital staff to manage patient records, doctor records and appointments. This SAD covers:

- Patient Management: register, search/view and update patients (UC-01 to UC-03).
- Doctor Management: add, search/view and update doctors (UC-04 to UC-06).
- Appointment Management: book, view, cancel and check availability, including doctor double-booking prevention (UC-07 to UC-10).
- Persistence in local CSV files, input validation, and safe handling of missing or malformed files.

Out of scope (as in the SRS): billing, pharmacy, laboratory, web/mobile front ends, database servers, networking, and multi-user concurrency. Optional features HMS-O-001 to HMS-O-004 are noted only as extension points (Section 4.6).

1.3 Audience

Project team members (developers and testers), the course instructor and evaluators.

1.4 Definitions

Term	Definition
HMS	Hospital Management System
SRS / SAD / STP / RTM	Software Requirements Specification / Software Architecture and Design Specification / Software Test Plan / Requirements Traceability Matrix
ADR	Architecture Decision Record
CSV	Comma-separated values text file
FR / NFR / SR / SO	Functional / Non-Functional / Security Requirement / Security Objective
UC / BR / VR / TC	Use Case / Business Rule / Validation Rule / Test Case
STL / OOP / UML	Standard Template Library / Object-Oriented Programming / Unified Modeling Language
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
Conflict	A second Booked appointment for the same doctor at exactly the same date and time (BR-05)

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (goals, components, pattern, technology, risks, traceability, security). Section 4 describes the detailed design (classes, sequence diagrams, interfaces, error handling, console UX, open issues). Section 5 contains appendices. Design decisions that are not dictated by the SRS are recorded as ADRs in Section 3.5 so that they can be reviewed and challenged.

2.2 Related Documents

Document	Identifier	Relationship
Software Requirements Specification	HMS SRS v1.0 (28-09-2026)	Source of all requirements, use cases, business rules (BR-01 to BR-11) and validation rules (VR-01 to VR-14)
Software Test Plan	STP-HMS-2026-V1.0	Verifies the design; defines test cases TC-PAT, TC-DOC, TC-APT, TC-PER, TC-UI, TC-NFR, TC-SEC
Requirements Traceability Matrix	RTM (STP Table 7; SAD Section 3.8)	Maps requirements to test cases and to design components

3. Architecture

3.1 Goals & Constraints

Architectural goals (derived from SRS Sections 5 and 6):

- Reliability and resilience: never crash on invalid keyboard input or missing/malformed data files (HMS-NF-001, NF-002).
- Data integrity: no record is changed unless all input is valid; a failed write never destroys existing data (BR-08, HMS-SR-005, HMS-F-034).
- Correctness: doctor double-booking detected with 100% accuracy (HMS-NF-008).
- Maintainability: five classes with file I/O isolated in one class (HMS-NF-006).
- Performance: startup ≤ 3 s and each operation ≤ 2 s with 1,000 records per file (HMS-NF-004).
- Privacy: minimum patient data on screen, fully offline operation (HMS-SO-03).

Constraints:

ID	Constraint	Architectural impact
C-01	C++ with OOP and the STL	Class-based design; std::vector, std::string, std::fstream
C-02	Console, menu-driven UI only	Presentation layer is a text menu loop
C-03	CSV / delimited text persistence only	No database; FileManager serialises entities to CSV
C-04	Single-user, single-process, no networking	No concurrency control, sockets or services
C-05	Comma-delimited fields	Commas rejected in all text fields (VR-14); no quoting or escaping is needed
NF-007	g++ (GCC 9+/MinGW-w64), -std=c++17, Windows 10+ and Linux	Standard library only; no platform-specific APIs

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed by
Hospital Staff / Administrator (sole actor)	Simple menus, clear messages, no lost data, correct booking conflicts	Sections 3.5, 4.4, 4.5
Developers	Modularity, clear class responsibilities, testable units	Sections 3.4, 4.1, 4.3
QA / Test engineers	Testability of each rule, traceability to test cases	Section 3.8; STP
Course instructor / evaluators	Completeness against SRS, UML deliverables, security design	Sections 3.3, 3.9, 4.2
Data owner (hospital)	Integrity and confidentiality of patient data on the local machine	Sections 3.9, 4.4

3.3 Component (UML) Diagram

Figure 1 shows the runtime components of HMS and their dependencies. Hospital Staff use the Console UI, which invokes operations on HospitalSystem. HospitalSystem manages the Patient, Doctor and Appointment entities and is the only component that talks to FileManager, which is the only component that reads or writes the three CSV files.

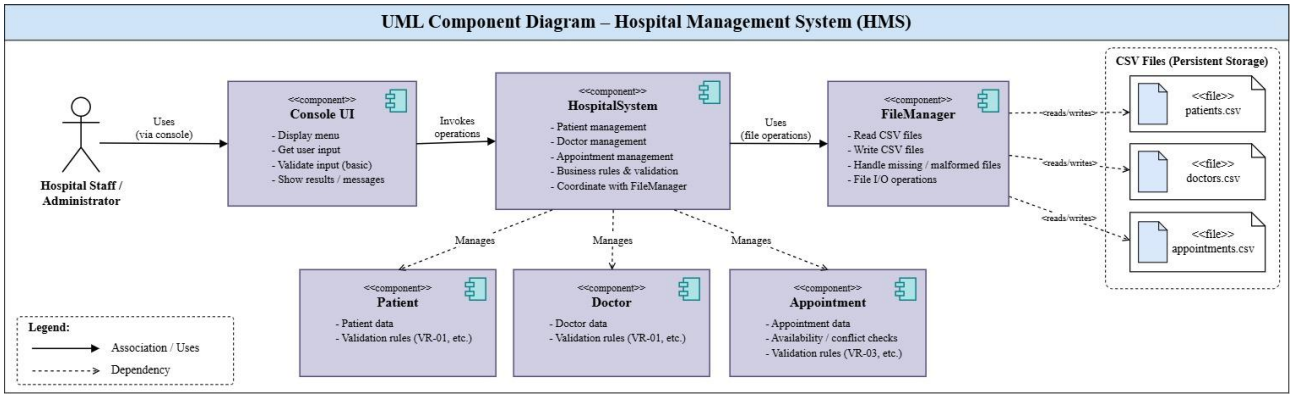


Figure 1: UML Component Diagram – Hospital Management System (HMS)

3.4 Component Descriptions

Component	Responsibilities	Depends on	Realises
Console UI	Displays main menu and submenus; reads input lines; performs basic input checks; shows results and error messages; re-prompts on invalid selections; ends the program on Exit.	HospitalSystem	HMS-F-032 to F-036, HMS-NF-001, NF-005
HospitalSystem	Central controller. Holds in-memory collections of Patient, Doctor and Appointment. Applies business rules (BR-01 to BR-06, BR-08), validates all input before changing state, checks appointment conflicts, generates Appointment IDs, coordinates FileManager saves.	Patient, Doctor, Appointment, FileManager	HMS-F-001 to F-024, F-034
Patient	Entity holding Patient ID, name, age, gender, contact, address and optional medical note; owns validation rules VR-01 to VR-07 and VR-14.	–	HMS-F-003, F-005; HMS-SR-001
Doctor	Entity holding Doctor ID, name, specialization, department, contact and availability text; owns validation rules VR-01, 02, 05, 08, 09, 14.	–	HMS-F-010, F-012; HMS-SR-001
Appointment	Entity holding Appointment ID, Patient ID, Doctor ID, date, time and status (Booked/Cancelled); owns rules VR-10 to VR-13 and the availability/conflict test.	–	HMS-F-015 to F-024, NF-008
FileManager	Only class that performs file I/O. Loads and saves the three CSV files, writes header rows, treats missing files as empty, skips malformed/duplicate/orphan lines and reports the count, writes files safely (temp file then replace), reports write failures.	Patient, Doctor, Appointment (for field validation)	HMS-F-025 to F-031; HMS-SR-004, SR-005, SR-010; HMS-NF-006, NF-010
CSV files (patients.csv, doctors.csv, appointments.csv)	Persistent storage in the data directory; header row, one record per line, comma separator, readable in a text editor.	–	HMS-NF-010

Data file layouts

File	Columns (in order)	Unique key
patients.csv	PatientID, Name, Age, Gender, Contact, Address, MedicalHistoryNote	PatientID
doctors.csv	DoctorID, Name, Specialization, Department, Contact, Availability	DoctorID
appointments.csv	AppointmentID, PatientID, DoctorID, Date, Time, Status	AppointmentID

3.5 Chosen Architecture Pattern and Rationale

HMS uses a layered architecture with three code layers on top of a flat-file storage layer:

Layer	Component(s)	Rule
Presentation	Console UI	Talks only to HospitalSystem; contains no business rules
Business / Domain	HospitalSystem, Patient, Doctor, Appointment	Contains every business and validation rule; never touches files directly
Data access	FileManager	Only class using file streams; knows CSV layout, not business rules (beyond reusing entity validators when loading)

Layer	Component(s)	Rule
Storage	patients.csv, doctors.csv, appointments.csv	Plain text; no database engine

Rationale: the layered style gives the clear separation of concerns required by HMS-NF-006, lets FileManager and Console UI be replaced by stubs during unit testing (STP Section 10), and matches the SRS logical flow User → Console UI → HospitalSystem → FileManager → CSV files. Microservices, client-server and MVC-with-views were rejected because C-04 excludes networking and concurrency, and the console has a single view.

Architecture Decision Records

ADR	Decision	Alternatives considered	Rationale / consequence
ADR-01	Layered architecture (UI → HospitalSystem → FileManager)	Monolithic single class; MVC; client-server	Separation of concerns, testability, matches SRS flow; small extra indirection
ADR-02	CSV flat files, one per entity	Single combined file; SQLite; binary files	Mandated by C-03; human-readable (NF-010); requires comma/newline rejection (VR-14)
ADR-03	Entire file rewritten after each modifying operation, via a temporary file that replaces the original only after a successful write	Append-only writes; in-place overwrite	Satisfies HMS-F-029 and HMS-SR-005 (previous content kept if write fails); O(n) cost is acceptable for ≤ 1,000 records
ADR-04	In-memory std::vector per entity loaded at startup; lookups by linear scan	Hash maps; on-demand file reads	Simple and well within the 2 s limit at 1,000 records (NF-004); avoids stale-file issues
ADR-05	Operations return a result object (success flag + message) instead of throwing across layers	Exceptions; error codes only	Console UI can always show exactly one success or error outcome (HMS-F-035) and internal details never leak (HMS-SR-009)
ADR-06	Cancellation sets status to Cancelled; no mandatory function deletes appointments	Physical delete	BR-06, HMS-SR-006 (audit integrity); conflict test ignores Cancelled (HMS-F-021)
ADR-07	Validation implemented once in entity classes and reused by both the UI input path and FileManager loading	Separate validators per layer	Single source of truth for Table 4-1; guarantees that files are treated as untrusted (HMS-SR-004)

3.6 Technology Stack & Data Stores

Area	Choice	Notes
Language / standard	C++17	Object-oriented; STL containers and strings (C-01)
Compiler / build	g++ (GCC 9+ / MinGW-w64), -std=c++17 -Wall	Windows 10+ and Ubuntu 22.04 (NF-007); no third-party libraries
UI	Text console, numbered menus (std::cin/std::cout, std::getline)	No colour dependence; plain text output
Data store	Three CSV files in a fixed data directory	std::fstream in text mode; header row; UTF-8/ASCII text
Data structures	std::vector<Patient>, std::vector<Doctor>, std::vector<Appointment>	Sorted with std::sort by date then time for views (HMS-F-022)

Area	Choice	Notes
Test tooling	PowerShell test harness, Measure-Command (STP Section 6)	Not part of the shipped executable
Deliverable	hms.exe (single executable)	Runs offline; only the compiler runtime is needed (NF-009)

3.7 Risks & Mitigations

#	Risk	Mitigation
1	A data file is edited outside HMS or corrupted, producing malformed, duplicate or orphan rows.	Every loaded field is validated with Table 4-1; bad rows are skipped and counted; duplicates and orphans are ignored (HMS-F-027, F-028, SR-004).
2	A write fails (read-only file, full disk, interruption) and destroys data.	Write to a temporary file first and replace the original only on success; on failure keep old content and show an error (ADR-03, HMS-F-031, SR-005).
3	Double-booking logic defect.	Single conflict function in Appointment/HospitalSystem; four-case conflict matrix test (HMS-NF-008, TC-NFR-08).
4	Over-long or hostile keyboard input causes crashes or CSV corruption.	Read whole lines into std::string, check length and character set before use; reject commas, line breaks and formula prefixes (HMS-SR-001 to SR-003).
5	Line-ending differences (\r\n vs \n) break parsing across Windows/Linux.	Text-mode streams; strip trailing \r on load; VR-14 forbids CR/LF in fields.
6	Slow operations as data grows.	Linear scans are sufficient for 1,000 records; profile early; containers can be swapped for maps without changing interfaces.
7	Two instances running on the same data directory overwrite each other.	Excluded by assumption A-01; documented as an operating rule.

3.8 Traceability to Requirements

Requirement(s)	Realised by (component / class)	Design section
HMS-F-001 to F-003 (register patient, duplicate ID, validation)	Console UI, HospitalSystem::registerPatient, Patient validators, FileManager::savePatients	4.2, 4.3
HMS-F-004, F-005 (search/view patient)	HospitalSystem::searchPatients / getPatient / listPatients	4.3
HMS-F-006, F-007 (update patient; ID immutable)	HospitalSystem::updatePatient (no ID parameter to change)	4.3
HMS-F-008 to F-014 (doctor add/search/view/update)	HospitalSystem doctor operations, Doctor validators, FileManager::saveDoctors	4.3
HMS-F-015 to F-018 (booking, FK checks, date/time validation)	HospitalSystem::bookAppointment, Appointment validators	4.2, 4.3
HMS-F-019 to F-021 (conflict, availability, cancelled excluded)	Appointment::conflictsWith, HospitalSystem::checkAvailability	4.2, 4.3
HMS-F-022 to F-024 (sorted views, cancel, invalid cancel)	HospitalSystem::listAppointments / cancelAppointment	4.3
HMS-F-025 to F-031 (load, missing file, malformed, duplicate/orphan, immediate save, restart, write error)	FileManager (load*/save*), HospitalSystem::loadAll	3.4, 4.3, 4.4
HMS-F-032 to F-036 (menus, invalid selection, state protection, messages, exit)	Console UI menu loop; HospitalSystem validate-then-commit rule	4.4, 4.5

Requirement(s)	Realised by (component / class)	Design section
HMS-NF-001, NF-002 (crash resilience)	Console UI input reader; FileManager tolerant loader	4.4
HMS-NF-004 (performance)	Vector-based in-memory model (ADR-04)	3.5
HMS-NF-006 (class isolation)	Five classes; file I/O only in FileManager	3.4, 4.1
HMS-NF-007, NF-009, NF-010 (portability, standalone, CSV format)	Standard C++17 only; CSV layouts in 3.4	3.4, 3.6
HMS-SR-001 to SR-003 (validation, injection guard, buffer safety)	Entity validators, Console UI line reader	3.9, 4.4
HMS-SR-004, SR-005 (untrusted files, non-destructive writes)	FileManager	3.9, 4.4
HMS-SR-006 (non-deletion)	HospitalSystem::cancelAppointment (status change only)	4.3
HMS-SR-007 to SR-010 (privacy, offline, no internals in errors, fixed data directory)	Console UI output formatting, no network code, generic messages, FileManager constant paths	3.9, 4.4

3.9 Security Architecture

HMS is a local, air-gapped, single-user program, so security focuses on integrity of stored records, safe handling of untrusted input and files, and minimal exposure of patient data (objectives HMS-SO-01 to SO-03). STRIDE analysis:

Threat	Scenario in HMS	Mitigation in the design	Requirement
Spoofing	Someone else uses the application on an unlocked machine	No in-app authentication in the baseline; access is controlled by operating-system login and folder permissions (A-05). Optional Administrator Login (UC-14) would store only a password hash and lock after 3 failures.	HMS-SR-011 (optional)
Tampering	Malformed input, formula/CSV injection, or hand-edited CSV files alter or corrupt records	Accept-list validation of every field; reject commas, CR/LF and leading = + - @; validate every loaded field and skip bad rows; validate-then-commit; temp-file-and-replace writes	HMS-SR-001, 002, 004, 005
Repudiation	A booking disappears so its history cannot be shown	Cancelled appointments stay in appointments.csv with status Cancelled; no mandatory function deletes appointments. (No per-user audit log: single anonymous actor.)	HMS-SR-006
Information disclosure	Patient contact, address or medical note exposed on screen, in errors, or over a network	Appointment views show only IDs and names; error messages contain no file paths, raw records or exception text; no network code; files only in the fixed data directory	HMS-SR-007, 008, 009, 010
Denial of service	Very long input or corrupted files crash the program	Line-based input into std::string with length checks (10,000-character test); tolerant loader that skips bad rows; every failure returns to a prompt	HMS-SR-003, 004; NF-001, NF-002
Elevation of privilege	User supplies a path or command to reach other files or run programs	No prompt accepts a file name or path; no external process execution; single role only	HMS-SR-010

Residual risks accepted for the baseline: anyone with operating-system access to the data directory can read or edit the plain-text CSV files, and files are not encrypted. These follow from constraints C-03 and C-04 and are mitigated by OS permissions and by treating all files as untrusted on load.

4. Design

4.1 Design Overview

HMS is built from five classes: Patient, Doctor, Appointment, HospitalSystem and FileManager, plus a console menu driver (Console UI) in the presentation layer. The guiding design rule is validate first, then commit: every operation validates all inputs and preconditions, and only then changes the in-memory collection and immediately rewrites the affected CSV file. If validation fails the operation returns an error and leaves memory and files unchanged (HMS-F-034).

Class design

Class	Key attributes	Key operations	Rules owned
Patient	patientId, name, age, gender, contact, address, medicalNote	static validate*(…) for each field; toCsv(); static fromCsv(line)	VR-01 to VR-07, VR-14; ID stored upper-case
Doctor	doctorId, name, specialization, department, contact, availability	static validate*(…); toCsv(); static fromCsv(line)	VR-01, 02, 05, 08, 09, 14
Appointment	appointmentId, patientId, doctorId, date, time, status	validateDate(); validateTime(); isBooked(); conflictsWith(doctorId, date, time); cancel(); toCsv(); fromCsv(line)	VR-10 to VR-13; BR-05 slot test (Booked only)
HospitalSystem	vector<Patient>, vector<Doctor>, vector<Appointment>, FileManager	loadAll(); patient, doctor and appointment operations (Section 4.3); nextAppointmentId()	BR-01 to BR-06, BR-08; VR-12 ID generation
FileManager	file names of the three CSV files (constants)	loadPatients/Doctors/Appointments(); savePatients/Doctors/Appointments()	BR-09, BR-10; HMS-SR-004, SR-005, SR-010
Console UI (menu driver)	none persistent	run(); mainMenu(); patientMenu(); doctorMenu(); appointmentMenu(); readLine(); readMenuChoice()	HMS-F-032 to F-036

Design details:

- IDs: Patient and Doctor IDs are entered by staff, compared case-insensitively and stored in upper case (VR-01). Appointment IDs are generated as "A" plus a zero-padded number of at least four digits, taking the highest existing number (including Cancelled records) plus one, so an ID is never reused (VR-12).
- Dates and times are stored as YYYY-MM-DD and HH:MM strings; this ordering makes a plain string sort equal to chronological order for the sorted views (HMS-F-022).
- Conflict test: an appointment conflicts if some Booked appointment has the same Doctor ID, date and time. Cancelled appointments are ignored. Different doctor or different time never conflicts (A-04).
- Doctor availability text (VR-09) is informational and is never used by the conflict test (A-03).
- Views resolve patient and doctor names by ID lookup and print only IDs and names (HMS-SR-007).

4.2 UML Sequence Diagrams

Figure 2 contains two sequence diagrams specific to HMS: (1) Register Patient (UC-01) and (2) Book Appointment with Availability Validation (UC-07 and UC-10), including the alternative flow when the doctor is not available.

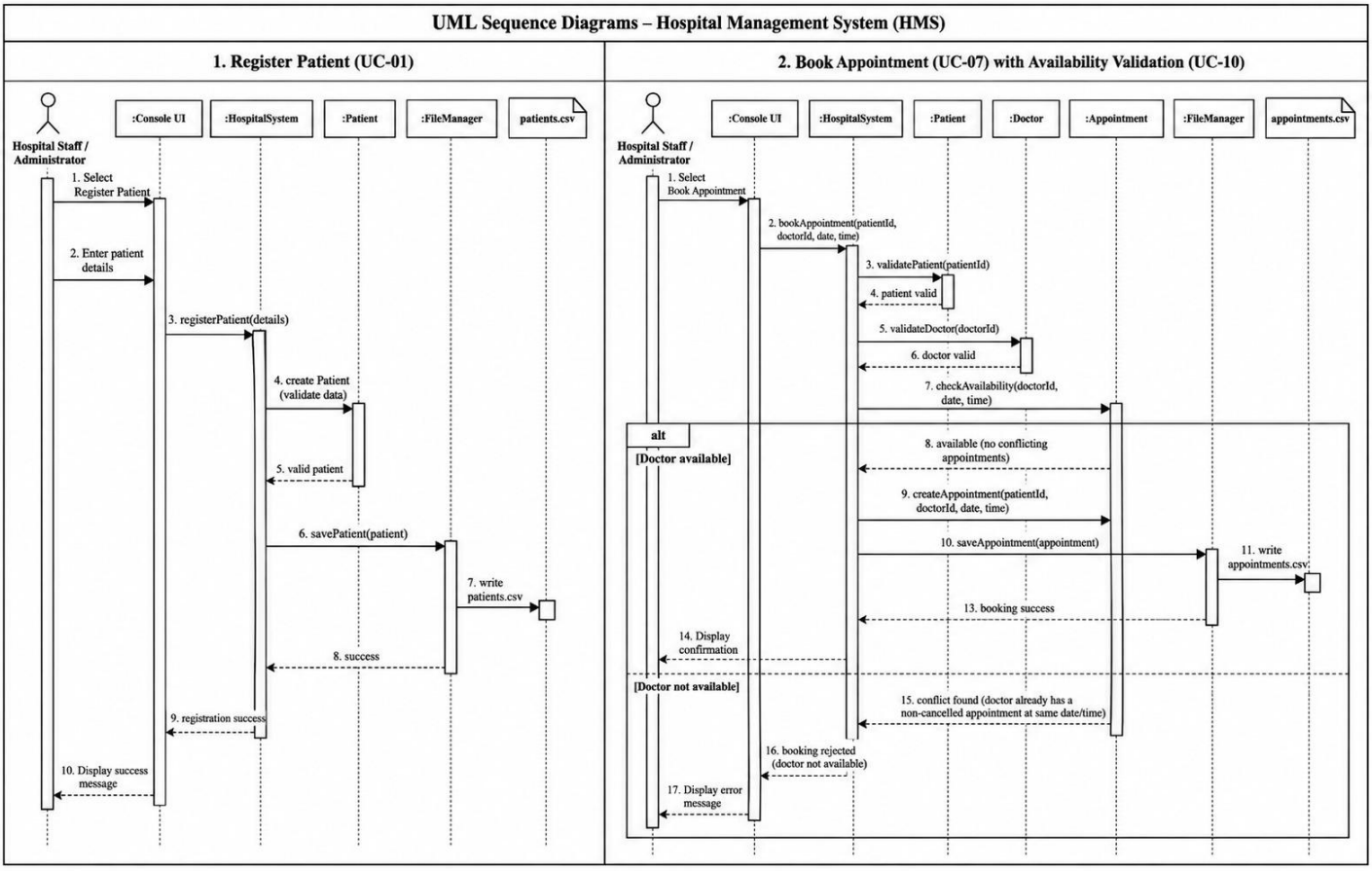


Figure 2: UML Sequence Diagrams – Register Patient (UC-01) and Book Appointment (UC-07/UC-10)

4.2.1 Register Patient (UC-01)

Hospital Staff select Register Patient and enter the patient details in the Console UI. The Console UI calls registerPatient(details) on HospitalSystem. HospitalSystem creates the Patient, which validates the data (VR-01 to VR-07, VR-14) and also checks that the Patient ID is not already used (BR-01). If valid, HospitalSystem asks FileManager to save (savePatient(patient) in the diagram; implemented as savePatients, which rewrites patients.csv from the in-memory list), and FileManager reports success; HospitalSystem returns "registration success" and the Console UI displays the confirmation. If a field is invalid or the ID exists, the operation stops before saving, nothing changes, and the Console UI shows the error (HMS-F-002, F-003). If the file cannot be written the error of HMS-F-031 is shown.

4.2.2 Book Appointment with availability validation (UC-07, UC-10)

The Console UI collects Patient ID, Doctor ID, date and time and calls bookAppointment(patientId, doctorId, date, time). HospitalSystem validates date and time (VR-10, VR-11), then validates that the patient and the doctor exist (BR-03, BR-04) and calls checkAvailability(doctorId, date, time). In the [Doctor available] branch, an Appointment with status Booked and a generated ID is created and saved through FileManager to appointments.csv, and the confirmation with the new Appointment ID is displayed. In the [Doctor not available] branch, a conflicting Booked appointment was found (BR-05), the booking is rejected with "Doctor not available at the requested date and time", no record is created, and the Console UI displays the error message (HMS-F-019).

Note: the diagram shows patient and doctor validation as calls to the Patient and Doctor components; in the implementation this is a lookup by ID in HospitalSystem's collections that returns "Patient not found" / "Doctor not found" (HMS-F-016, F-017).

4.3 API Design

HMS has no network API. Its interfaces are the C++ public interfaces between components. Two are specified in full: HospitalSystem (used by Console UI) and FileManager (used by HospitalSystem). Both use the shared result type below (ADR-05).

Shared types

Type	Definition	Purpose
OpResult	struct { bool success; std::string message; }	Outcome of every modifying operation; message is the text shown to staff (success confirmation or error reason)
LoadReport	struct { int loaded; int skipped; }	Returned by FileManager loads; skipped is displayed as "N malformed record(s) skipped"
Status	enum { Booked, Cancelled }	Appointment status (VR-13)

Interface 1 – HospitalSystem (called by Console UI)

Operation	Inputs	Result / errors	Req.
loadAll()	–	Loads all three files via FileManager; returns LoadReport per file; never fails fatally	F-025 to F-028
registerPatient(...)	id, name, age, gender, contact, address, note	OpResult. Errors: field + rule violated; "Patient ID already exists"; save-failure message	F-001 to F-003
searchPatients(mode, value)	mode = ID (exact) or Name (case-insensitive partial)	vector<Patient>; empty → "No matching patient found"	F-004

Operation	Inputs	Result / errors	Req.
getPatient(id) / listPatients()	ID	Patient record with all fields / list of ID, name, age, gender, contact; empty → "No patients registered"	F-005
updatePatient(...)	id, name, age, gender, contact, address, note (ID cannot be changed)	OpResult. Errors: "Patient not found"; validation errors; record unchanged on any error	F-006, F-007
addDoctor(...)	id, name, specialization, department, contact, availability	OpResult. Errors: "Doctor ID already exists"; validation errors	F-008 to F-010
searchDoctors(mode, value) / getDoctor(id) / listDoctors()	mode = ID, Name or Specialization	Matching doctors; "No matching doctor found"; "No doctors registered"	F-011, F-012
updateDoctor(...)	id (fixed), name, specialization, department, contact, availability	OpResult. Errors: "Doctor not found"; validation errors	F-013, F-014
bookAppointment(patientId, doctorId, date, time)	IDs, date YYYY-MM-DD, time HH:MM	OpResult with new Appointment ID. Errors: "Patient not found", "Doctor not found", date/time format errors, "Doctor not available at the requested date and time"	F-015 to F-019
checkAvailability(doctorId, date, time)	doctor ID, date, time	"Available" or "Not Available"; changes nothing	F-020, F-021
listAppointments(filter)	filter = all, by Patient ID, or by Doctor ID	Appointments sorted by date then time, showing IDs, names, date, time, status only; empty → "No appointments found"	F-022; SR-007
cancelAppointment(appointmentId, confirmed)	ID, staff confirmation	OpResult. Sets status Cancelled (record kept). Errors: "Appointment not found", "Appointment already cancelled"; nothing changes if not confirmed	F-023, F-024; SR-006

Interface 2 – FileManager (called by HospitalSystem)

Operation	Behaviour	Errors / edge cases	Req.
loadPatients(vector<Patient>&) loadDoctors(vector<Doctor>&) loadAppointments(vector<Appointment>&, patients, doctors)	Reads the fixed-name CSV file, skips the header, parses each line, validates every field with the entity validators, appends valid records and returns LoadReport.	Missing file → empty dataset, no error. Wrong field count, invalid field, duplicate ID, or orphan appointment → line skipped and counted. Trailing \r removed. Program never terminates.	F-025 to F-028; SR-004; BR-09, BR-10
savePatients(const vector<Patient>&) saveDoctors(...) saveAppointments(...)	Writes header and all records to a temporary file in the data directory, then replaces the original file with it; creates the file if it did not exist.	Any stream or replace failure → original file left untouched; returns OpResult with a generic message such as "Changes may not have been saved" (no paths or system text).	F-029, F-031; SR-005, SR-009

Interface 3 – Console UI menu contract

Menu	Options
Main menu	1 Patient Management; 2 Doctor Management; 3 Appointment Management; 4 Exit
Patient Management	1 Register Patient; 2 Search Patient; 3 View All Patients; 4 Update Patient; 0 Return
Doctor Management	1 Add Doctor; 2 Search Doctor; 3 View All Doctors; 4 Update Doctor; 0 Return

Menu	Options
Appointment Management	1 Book Appointment; 2 View Appointments; 3 Cancel Appointment; 4 Check Doctor Availability; 0 Return

4.4 Error Handling, Logging & Monitoring

Error handling

- Every operation ends with exactly one outcome: a success confirmation or an error message stating the reason (HMS-F-035). Error messages name the field and the rule violated (HMS-NF-005).
- Invalid menu selections (empty, non-numeric, out of range) produce an error and the same menu is shown again (HMS-F-033).
- Input is read as a whole line with std::getline into std::string, then checked for length, character set and format before use; over-length input (for example 10,000 characters) is rejected, not truncated silently (HMS-SR-001, SR-003).
- Validation failures never modify memory or files. Modifying operations first validate, then update memory, then save; if the save fails, the user is told that changes may not have been saved and the program keeps running (HMS-F-031, F-034).
- Startup never fails because of data files: missing files are empty datasets; malformed, duplicate and orphan rows are skipped and counted per file (HMS-F-026 to F-028, NF-002).
- Error messages contain no file-system paths, raw record contents or exception text (HMS-SR-009). Exceptions from the standard library, if any, are caught inside FileManager/HospitalSystem and converted to generic messages.

Standard messages

Situation	Message text
Duplicate patient / doctor ID	Patient ID already exists / Doctor ID already exists
Unknown patient / doctor / appointment	Patient not found / Doctor not found / Appointment not found
Slot taken	Doctor not available at the requested date and time
Already cancelled	Appointment already cancelled
No results	No matching patient found / No matching doctor found / No appointments found / No patients registered / No doctors registered
Malformed rows on load	<n> malformed record(s) skipped
Illegal character	Input contains illegal character (comma, newline, or formula prefix =, +, -, @)

Logging and monitoring

The baseline writes no log file: an additional file would create another store of patient-related data outside the SRS scope. Feedback is given on the console (messages above and the per-file skipped-record count at startup). No sensitive data appears in any message. Monitoring of quality is done at test time rather than at run time, using the STP measures: process exit code and crash count for the 20 invalid-input vectors (TC-NFR-01), Measure-Command timings for startup and operations (TC-NFR-04), and file comparison before/after operations (HMS-F-020, F-034).

4.5 UX Design

- Numbered text menus (Section 4.3, Interface 3); the user types the number and presses Enter. Every operation is reachable within at most 3 selections from the main menu (HMS-NF-005).
- Every prompt states the expected format, for example "Enter date (YYYY-MM-DD):" and "Enter time (HH:MM, 24-hour):"; on an invalid value only that field is re-prompted (UC-01 exception flow 4a).
- Output is plain text with aligned columns so it is readable in any console; no colour is required to understand a message.
- Destructive-looking actions ask for confirmation: cancelling an appointment requires staff confirmation and declining changes nothing (HMS-F-023).
- Privacy on screen: appointment lists show Appointment ID, Patient ID and name, Doctor ID and name, date, time and status only; contact, address and medical note appear only in the single-patient detail view (HMS-SR-007).
- Exit (main menu option 4) ends the program normally with exit code 0, all confirmed changes already stored (HMS-F-036).

4.6 Open Issues & Next Steps

#	Item	Notes
1	Optional features HMS-O-001 to O-004 (delete patient, reschedule, basic report, administrator login)	Not in the mandatory baseline. Extension points: new HospitalSystem operations reusing bookAppointment validation (O-002) and a hashed-credential check before the main menu (O-004/SR-011).
2	SAD/SRS/STP consistency	SRS defines security requirements HMS-SR-001 to SR-010 (SR-009 and SR-010 cover error-message content and fixed data directory); STP Table 7 and Section 5.1 list only SR-001 to SR-008. Add TC-SEC-09/10 coverage to the STP RTM so all 10 mandatory security requirements are traced.
3	Test case ID for HMS-F-021	SRS names TC-APT-07; STP Table 7 maps HMS-F-021 to TC-APT-05. Align the identifier.
4	Run-time audit log	Not provided (see 4.4). Could be added later if the scope allows a fourth file.
5	Data at rest	CSV files are unencrypted; consider file-level protection only if the scope changes.
6	Concurrent use	Explicitly unsupported (A-01); a lock file could detect a second instance in future.

5. Appendices

5.1 Glossary

HMS – Hospital Management System; SRS – Software Requirements Specification; SAD – Software Architecture and Design Specification; STP – Software Test Plan; RTM – Requirements Traceability Matrix; ADR – Architecture Decision Record; CSV – comma-separated values; FR/NFR/SR/SO – functional, non-functional, security requirement and security objective; UC – use case; BR – business rule; VR – validation rule; STL – Standard Template Library; STRIDE – Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.

5.2 References

- HMS Software Requirements Specification (SRS) v1.0, 28-09-2026.
- HMS Software Test Plan (STP-HMS-2026-V1.0), 28-09-2026.

- Software Engineering Mini-Project Deliverables and SAD template, course materials.
- ISO/IEC/IEEE 42010, Systems and software engineering – Architecture description.
- ISO/IEC/IEEE 29148:2018, Requirements engineering; IEEE Std 830-1998; IEEE Std 829-2008.
- OWASP guidance on input validation and CSV (formula) injection.

5.3 Tools

draw.io / diagrams.net (UML component and sequence diagrams), g++ / MinGW-w64 (build), PowerShell with Measure-Command (test harness), Visual Studio Code (development).